

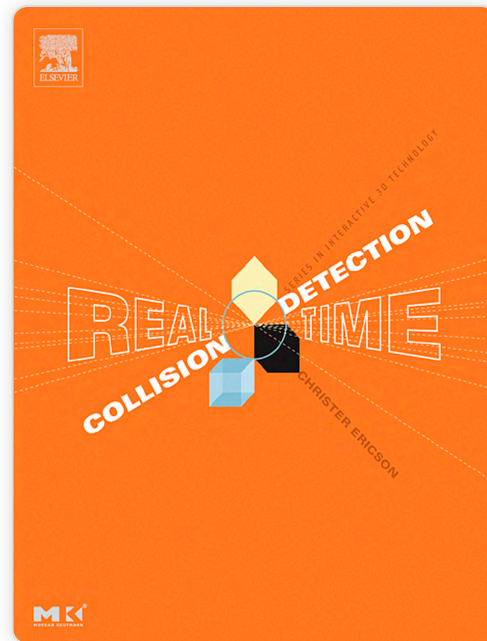
碰撞检测初步

关键词：#碰撞检测 #物体表示 #包围盒 #几何碰撞 # 包围盒层次

2025-10-11 @矩阵前端研发二组

简要

- 碰撞检测设计问题
- 物体表示
- 包围盒
- 基础几何碰撞
- 包围盒层次结构



作者: 金观涛

碰撞检测

碰撞检测关注的是两个物体 是否 (if) 在 什么时候 (when) 以及 在哪里 (where) 发生碰撞。

碰撞检测设计问题 - 考虑因素

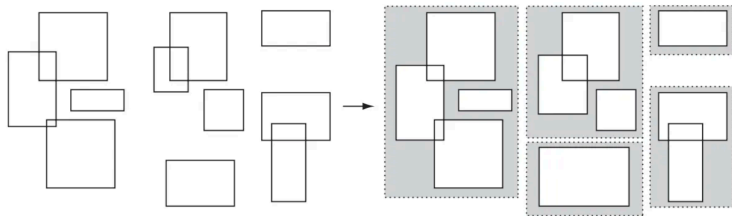
- **应用领域的数据表示**：比如环境中物体的表示方式。
- **不同种类的查询**：两个物体是否碰撞，何时碰撞，在哪里发生碰撞。
- **环境参数**：物体是否运动，环境中有多少物体，这些物体是否允许穿透。
- **性能**：是否需要实时监测，需要的精度。
- **健壮性**
- **使用和实现的难度**

碰撞检测设计问题 - 查询种类

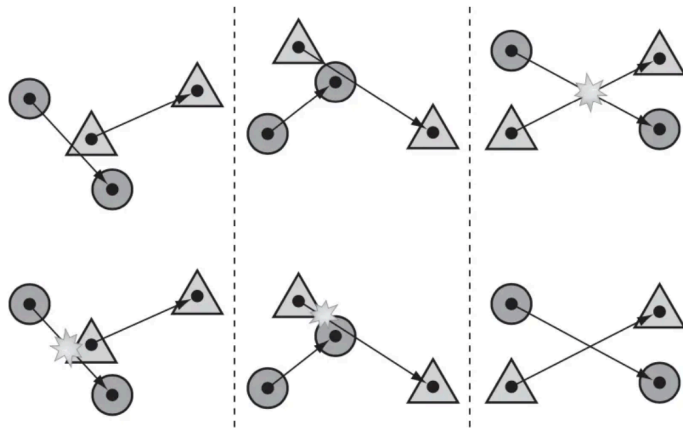
- **碰撞测试**：仅需要判断两个物体是否发生了碰撞，是回答“是否”的问题，计算复杂度较低。
- **碰撞点搜索**：需要找出一个或多个碰撞点，如果是两个立方体平面碰撞，会涉及到无数个碰撞点，对于两个凸多面体的碰撞，可能会有一个或多个碰撞点，对于凹多面体，还会有相互穿刺的可能，此时还需要判断碰撞点是否在几何体内部。
- **穿刺查询**：如果两个物体发生了穿刺，有时候还需要计算刺穿深度，这个深度是用来解决刺穿问题。
- **距离**：计算两个物体之间的最近距离，用于做一些预判。如果两个物体都是基础几何体，那距离计算较为简单，但如果都是复杂的凸多面体，需要精确到点到点之间的距离。
- **近似查询**：上面各种查询，都可以通过一个容忍参数来控制检测精度，从而提升性能。

碰撞检测设计问题 - 环境

- **物体数量**：碰撞检测是物体之间的两两检测，如果环境中有 n 个物体，那么碰撞检测次数为 $(n - 1) + (n - 2) + \cdots + 1 = n(n - 1)/2 = O(n^2)$ ，复杂度较高，所以一般碰撞检测会分为两个阶段，**粗略阶段 (broad phase)** 和**细致阶段 (narrow-phase)**。



- **动作同步检测与顺序检测**：在一个时间步里，环境中的物体会发生位移，同时处理这些位移以及顺序处理这些位移，是会得到不同的结果的。



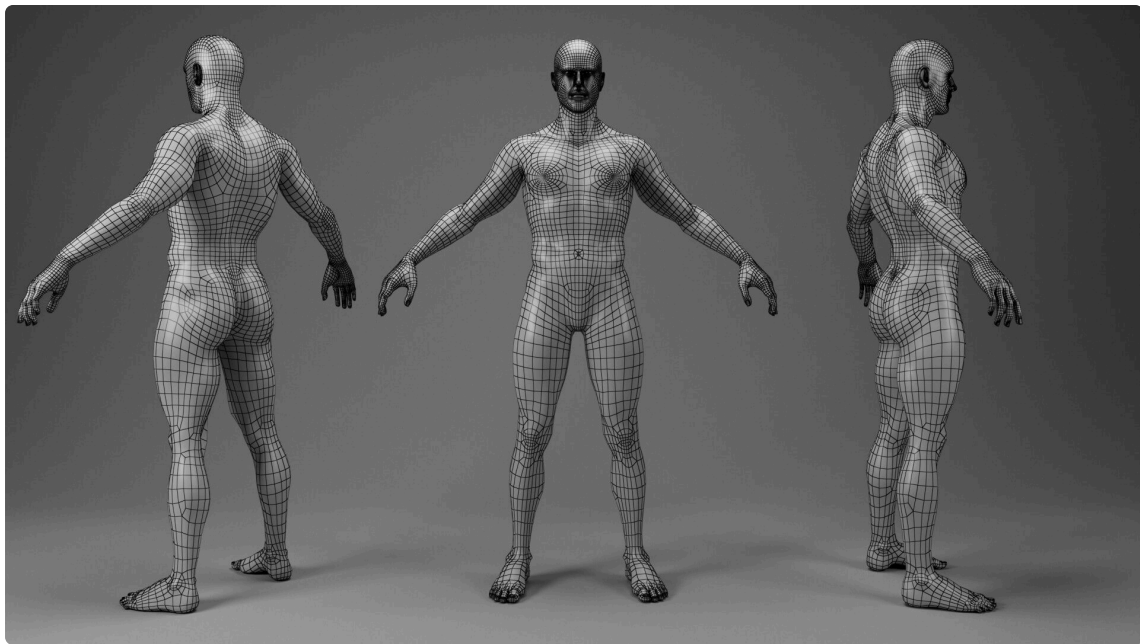
- **离散运动与连续运动**：静态碰撞检测只检测两个物体发生碰撞的情况，动态检测是指，物体是运动的，在物体运动过程中会**扫过一个体积 (swept volume)**，检测物体之间扫过体积的碰撞情况，是一个更为复杂和困难的问题。

碰撞检测设计问题 - 性能

一般比较好的影视作品或者游戏都要求 60fps，一秒 60 帧意味着一帧的时间大概是 $1s/60 = 16.7ms$ ，根据经验，一般游戏的碰撞检测计算量占一帧时间的 10-30%，也就是留给碰撞检测的时间只有 2-5ms 左右。不过这是一个比较明确的性能目标，碰撞检测可以根据这个目标进行优化。

物体表示

在计算机图形学里，物体皆为**网格（mesh）**。而网格是由一组**三角形（triangle）**组成。对于任何图形来说，都由**顶点（vertex）**和**线（edge）**组成，而物体的这个组成一般称为**物体的几何结构（geometry）**。



物体表示

一个物体在**环境（world）**中有自己的**位置（position）**，以及自己的**变换（transformation）**。位置是一个向量，而变换，则是一个矩阵。

```
1  class Object {  
2      position: Vector3  
3      transformation: Matrix4  
4      geometry: Geometry  
5  }  
6  
7  class Geometry {  
8      triangles: Triangle[]  
9  }  
10  
11 class Triangle {  
12     vertices: Vertex[3]  
13     edges: Edge  
14 }
```

物体表示 - 碰撞检测结构

实际进行碰撞检测的时候，我们需要知道在具体时刻物体的具体位置，需要进行碰撞检测的两个物体必须在同一个坐标系下才能够进行判断，此时需要将物体变换到同样的坐标系下，一般会在世界坐标系下进行判断。

```
1  const obj1 = new Object()  
2  const obj2 = new Object()  
3  
4  const o1 = obj1.applyTransformation()  
5  const o2 = obj2.applyTransformation()
```

经过变换之后两个物体都在同一坐标系下，此时就可以根据各种需要对这两个对象进行碰撞检测了。不同的查询，不同的精度，以及不同的算法，都会使用到不同的数据。

```
1  const o1Triangles = o1.geometry.triangles  
2  const o1Vertices = _.flatten(_.map(o1.geometry.triangles, 'vertices'))
```

包围盒

实际生产中物体可能非常复杂，尤其对于各种 3A 大作来说模型的精度很高，面片数量，顶点数量非常巨大，每次检测都遍历这些顶点非常不现实，一般会根据场景需要进行适度简化。

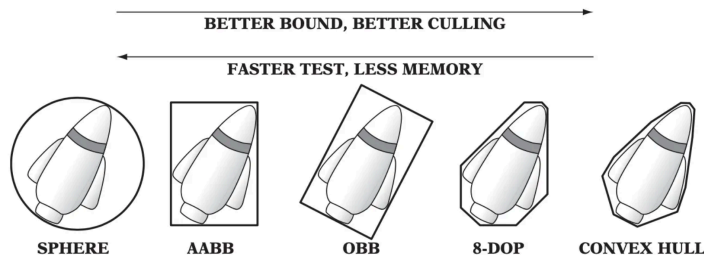
- 对于陨石撞地球这个场景，不一定需要做到精准的陨石表面碰到地球表面的山河湖海才判定为撞击，因为撞击的同时会有一个爆炸动画，所以撞击点的判定不需要过于精确。（可以简化为**球体与球体的碰撞**）
- 在一些射击游戏中，主要玩法是第一人称的射击，对于玩家之间的碰撞，也不需要非常精确，所以很多时候会看到一些刺穿，或者同组玩家可以直接穿过。（可以简化为**立方体与立方体的碰撞**）

从上面两个例子可以看出，通过为物体赋予一个**包围盒 (bounding box)**，尤其是**规则几何形状**的包围盒，可以大大简化碰撞检测的计算。

包围盒 - 不同种类的包围盒

根据使用场景，有不同的包围盒可以选择，当然，不同的包围盒会有不同的优缺点，下面是使用包围盒的一些考虑点。

- 简便的碰撞测试计算
- 与模型本身的贴合度
- 计算量
- 变换的便捷性
- 内存的使用



上图展示了五种包围盒，各有各的特点。

- 左边的包围盒结构简单，计算简单，快捷
- 右边的包围盒与模型的贴合度更高，但计算较为复杂

由此，物体之间的碰撞检测，便转化为基础几何的碰撞检测。

基础几何碰撞 - 架构与思路

对于复杂模型的碰撞检测，便转化为基础几何结构之间的碰撞检测。所以在碰撞检测前，我们根据需要先计算物体的包围盒，然后使用对应类型的碰撞算法来计算包围盒的碰撞情况，当然，跟查询也有关系。

- 如果查询是否碰撞，计算量会更为简单
- 如果查询距离，那计算会稍微复杂点
- 如果计算准确的碰撞点，也会更为复杂。

计算流程为

1. 计算物体包围盒
2. 根据查询类型选择合适的包围盒碰撞算法
3. 得到结果

```
1 // 将物体转换到同一个坐标系下
2 const o1 = obj1.applyTransformation()
3 const o2 = obj2.applyTransformation()
4
5 // 分别计算两个物体的包围盒
6 const bb1 = o1.getBoundingBox()
7 const bb2 = o2.getBoundingBox()
8
9 // 计算包围盒是否相交
10 const result = intersectionTest(bb1, bb2)
```

基础几何碰撞 - 架构与思路

前面提到的五种包围盒，球体，AABB，OBB，8-DOP，凸包，理论上需要实现两两匹配的碰撞算法。对于不同的查询，理论上可以在一个方法里面实现，比如

```
1  const intersect = (bb1: BoundingBox, bb2: BoundingBox): {  
2    // 是否碰撞  
3    intersected: boolean,  
4    // 两个物体的最近距离  
5    distance: number,  
6    // 碰撞点的世界坐标  
7    intersectionPoint: Point3,  
8  } => {  
9    // 一番计算  
10 }
```

但一般为了性能考虑，会分成三个子函数方便应用多维度使用。

```
1  type testIntersection = (bb1: BoundingBox, bb2: BoundingBox): boolean  
2  type getDistance = (bb1: BoundingBox, bb2: BoundingBox): number  
3  type getIntersectionPoint = (bb1: BoundingBox, bb2: BoundingBox): Point3
```

基础几何碰撞 - 架构与思路

所以需要对所有的包围盒进行两两配对，分别实现碰撞算法。在前述列出的所有包围盒的前提下，需要实现

```
1  // 球体与球体之间的碰撞检测
2  type testSphereIntersection = (bb1: Sphere, bb2: Sphere): boolean
3  type getSphereDistance = (bb1: Sphere, bb2: Sphere): number
4  type getSphereIntersectionPoint = (bb1: Sphere, bb2: Sphere): Point3
5
6  // 球体与AABB之间的碰撞检测
7  type testSphereAABBIntersection = (bb1: Sphere, bb2: AABB): boolean
8  type getSphereAABBDistance = (bb1: Sphere, bb2: AABB): number
9  type getSphereAABBIntersectionPoint = (bb1: Sphere, bb2: AABB): Point3
10
11 // 如此类推
```

其中各种包围盒类型需要继承包围盒基类，这样便可以把各种包围盒相关的方法封装到基类种中。

```
1  class Sphere extends BoundingBox {}
2  class AABB extends BoundingBox {}
```

基础几何碰撞 - 一个简单的例子

下面我们通过一个简单的例子，说明上面三种查询的计算方法。最简单的包围盒莫过于球体，而球体的数学结构极其简单。只需要一个球心以及半径即可表示。

```
1  class Sphere extends BoundingBox {  
2      center: Point3  
3      radius: number  
4  }
```

碰撞检测：判断两个球是否碰撞，只需要看球心距离是否大于半径之和即可。

```
1  const getSphereAABBDistance = (bb1: Sphere, bb2: AABB): number => {  
2      const c1 = bb1.center  
3      const c2 = bb2.center  
4  
5      const distance = c1.sub(c2).distance()  
6      return distance - bb1.radius - bb2.radius  
7  }
```


基础几何碰撞 - 一个简单的例子

距离计算： 计算两个球之间的距离，只需要计算两个球心之间的距离，然后减去半径之和即可。

```
1  const testSphereIntersection = (bb1: Sphere, bb2: Sphere): boolean => {
2      const c1 = bb1.center
3      const c2 = bb2.center
4
5      const distance = c1.sub(c2).distance()
6      return distance <= bb1.radius + bb2.radius
7  }
```

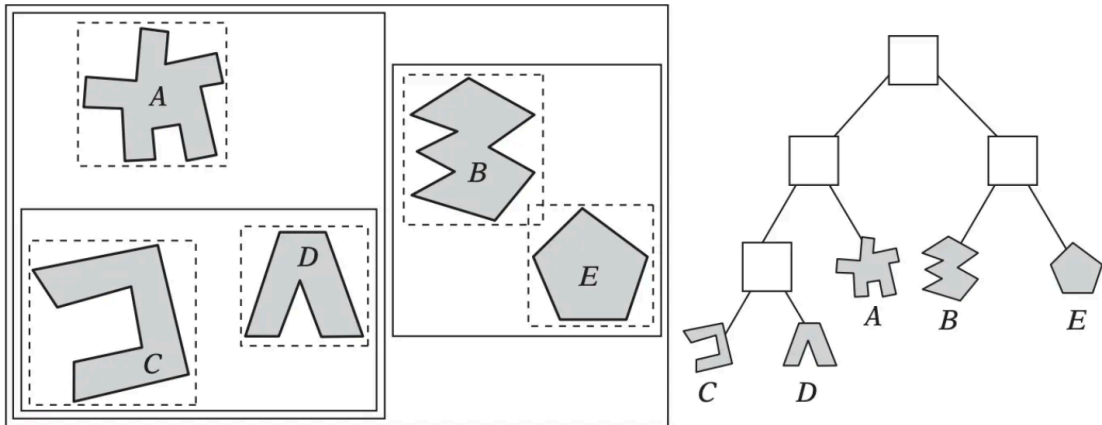
碰撞点计算： 如果两个球体发生碰撞，碰撞点就在球面上，方向是球心连线的方向，所以计算方法是取其中一个球的球心，加上球心方向的半径即可。

```
1  const getSphereAABBIntersectionPoint = (bb1: Sphere, bb2: AABB): Point3 => {
2      const c1 = bb1.center
3      const c2 = bb2.center
4
5      const centerVector = c1.sub(c2)
6      const radiusVector = centerVector.mul(c2.radius / centerVector.len())
7      return c2.add(radiusVector)
8  }
```

包围盒层次

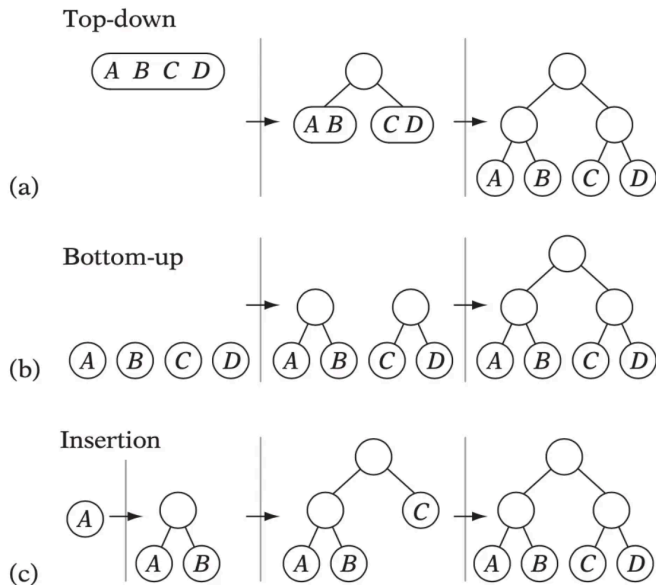
包围盒概念可以用于不同层次，是一个递归概念。把包围盒通过一棵树组织起来，就是一个 **包围盒层次结构 (BVH, bounding volume hierarchy)** 对于场景来说一般是两个层次。

- 对物体进行包围盒层次构建。比如一个角色，四肢分别用OBB包围盒包裹，头部用球体包裹，躯干用AABB包裹。然后整个角色被一个AABB包裹，这样得到下面的层次结构
- 对环境进行包围盒层次构建，把距离较近的物体放到一个包围盒里。



包围盒层次 - 构建

构建包围盒有三种方式，**自顶向下**，**自底向上**，**相交构建**。



下面用自顶向下的构建方式稍作说明。

```
1  const constructBVH = (objects: Object[]) => {  
2      // 计算根节点  
3      const bvh = new BVH()  
4      bvh.bb = computeBoundingBox(objects)  
5  
6      // 如果物体数量已经小于叶子的物体数量限制，就不用再分割了  
7      if (objects.length < MIN_OBJECTS_PER_LEAF) {  
8          bvh.type = BVH.LEAF  
9          bvh.objects = objects  
10         return  
11     }  
12  
13     bvh.type = BVH.NODE  
14     // 将物体根据一定规则分为两部分  
15     const [objs1, objs2] = partitionObjects(objects)  
16     // 构建 BVH 左右子树  
17     bvh.left = constructBVH(objs1)  
18     bvh.right = constructBVH(objs2)  
19  
20     return bvh  
21 }
```

包围盒层次 - 碰撞检测

有了 BVH，碰撞检测的时候复杂度从 $O(N^2)$ 降到 $O(\log N)$ 。判断的时候逐层判断，由于 BVH 每一个节点都是一个标准的包围盒，使用对应的碰撞算法可以得到碰撞情况。

```
1  const testBVH = (h1: BVH, h2: BVH): boolean => {
2    // 如果两个子树没有相交，那么就没有相交
3    if (!testIntersection(h1.bb, h2.bb)) return false
4
5    // 两个都是叶子节点，需要对叶子中的物体两两进行碰撞检测
6    if (h1.type === BVH.LEAF && h2.type === BVH.LEAF) {
7      return testPrimitives(h1.objects, h2.objects)
8    }
9
10   if (descendH1(h1, h2)) {
11     return testBVH(h1.left, h2) || testBVH(h1.right, h2)
12   } else {
13     return testBVH(h1, h2.left) || testBVH(h1, h2.right)
14   }
15 }
```

小结

- 碰撞检测用于回答两个物体 **是否碰撞**，**何时碰撞**，以及 **碰撞点的位置**。
- 碰撞检测分两个阶段，**粗略阶段** 和 **细致阶段**。
- 物体表示一般以网格形式，本质是一堆三角形，进一步的本质是顶点与边的集合。
- 通过选择合适的包围盒可以简化物体碰撞检测。
- **包围盒（BB, bouding box 或 BV, bouding volume）** 分为 **球体**，**AABB**，**OBB**，**8-DOP** 以及 **凸包**。
- 包围盒碰撞检测可以枚举为不同类型的包围盒之间的两两检测。
- **包围盒层次（BVH）** 本质是二叉树，对空间物体，或者对物体内部进行分割而构建的二叉树。

Q & A